

УЧЕБНАЯ ПРАКТИКА УП.08

«Разработка дизайна веб-приложений»

МДК.08.02 Графический дизайн и мультимедиа · ПК 8.2, ПК 8.3 · 72 часа

Задания на учебную практику

1. Общая идея практики

Вместо набора разрозненных упражнений каждый студент (или команда из 2–3 человек) за время практики проходит **полный цикл создания реального веб-приложения**: от технического задания и макета до работающего сайта с базой данных, который открывается по публичной ссылке с любого телефона.

Каждое задание практики — это очередной этап одного проекта. Все семь видов работ из рабочей программы (ТЗ и структура, макет, CSS, гиперссылки и меню, таблицы, формы, анимация) выполняются не «на пустой странице», а внутри вашего приложения. К концу практики у вас будет готовый проект для портфолио и резюме.

Главные правила

1. **Не чистый HTML.** Сайт из статичных .html-файлов не принимается. Интерфейс собирается на фреймворке (React, Vue, Svelte и др.), данные приходят с сервера.
2. **Есть база данных.** Все данные (товары, записи, пользователи, заявки) хранятся в БД, а не прописаны в коде.
3. **Есть сервер (бэкенд)** на любом языке программирования: Python, JavaScript/TypeScript, PHP, C#, Go, Java.
4. **Всё лежит на GitHub.** Работа ведётся в репозитории с регулярными коммитами.
5. **Проект опубликован в интернете.** На защите проект открывается по ссылке, а не с флешки.
6. Конструкторы сайтов (Tilda, Wix, «Конструктор сайтов» и т. п.) и готовые шаблоны «один в один» запрещены.

2. Технологический стек

Стек выбирается самостоятельно из таблицы ниже (можно предложить свой вариант — по согласованию с руководителем практики). Выбор нужно обосновать в техническом задании.

Слой	Варианты (на выбор)	Минимум для зачёта
Макет / дизайн	Figma, Pixso, Penpot	UI-kit + макеты desktop и mobile



Слой	Варианты (на выбор)	Минимум для зачёта
Фронтенд	React, Vue 3, Svelte, Angular; сборщик Vite; можно TypeScript	SPA или SSR на фреймворке
Стили	SCSS/Sass, Tailwind CSS, CSS Modules, CSS-переменные	Адаптив, Flexbox и Grid
Бэкенд	Python (FastAPI, Django, Flask); Node.js (Express, NestJS); PHP (Laravel); C# (ASP.NET Core); Go (Gin, Fiber); Java (Spring Boot); fullstack: Next.js, Nuxt	REST API: чтение, создание, изменение, удаление (CRUD)
База данных	PostgreSQL, MySQL/MariaDB, SQLite, MongoDB; ORM: Prisma, SQLAlchemy, Eloquent, Entity Framework	Не меньше 3 связанных таблиц
Контроль версий	Git + GitHub (альтернативы: GitFlic, GitVerse)	Не менее 15 осмысленных коммитов
Хостинг (деплой)	Фронтенд: GitHub Pages, Vercel, Netlify. Бэкенд и БД: Render, Railway, Amvera, Timeweb Cloud, Supabase, Neon. Или собственный VPS	Рабочая публичная ссылка
Анимация	CSS-анимации, Swiper, GSAP, Framer Motion, AOS, Lottie	Слайдер + анимации при скролле

Примечание: условия бесплатных тарифов и доступность хостингов могут меняться — актуальность нужно проверить на момент начала практики.

3. Темы проектов

Выберите одну тему (темы в одной группе не повторяются) или предложите свою. В колонке «Фишка» — обязательная особенность, которая делает проект интересным.

№	Тема	Фишка проекта
1	Кинотеатр с онлайн-бронированием	Интерактивная схема зала: занятые места подгружаются из БД, выбор мест кликом
2	Барбершоп / салон красоты	Запись к мастеру по свободным слотам времени, личный кабинет клиента
3	Кофейня с онлайн-заказом	Конструктор напитка (объём, сироп, молоко) с пересчётом цены, корзина
4	Портфолио-платформа студентов-дизайнеров	Загрузка работ, лайки, фильтр по направлениям (UI, 3D, иллюстрация)
5	Библиотека колледжа	Каталог с поиском, бронирование книги, статус «на руках / свободна»
6	Поиск команды для киберспортивных турниров	Анкеты игроков, фильтр по игре и рангу, заявки в команду



№	Тема	Фишка проекта
7	Сервис рецептов «Что в холодильнике»	Подбор рецептов по выбранным продуктам, избранное
8	Квест-комнаты	Бронирование, таблица рекордов прохождения, отзывы
9	Приют для животных	Карточки питомцев из БД, анкета «Хочу забрать домой», фильтры
10	Трекер привычек с геймификацией	Опыт, уровни и ачивки за выполнение привычек, графики прогресса
11	Фитнес-клуб	Расписание тренировок, запись на занятие с лимитом мест, абонементы
12	Барахолка колледжа	Объявления с фото, категории, чат-заявка продавцу
13	Гид по родному городу	Карта с метками (API Яндекс.Карт или Leaflet), маршруты, рейтинг мест
14	Магазин мерча / стритвира	Каталог с фильтрами по размеру и цвету, корзина, оформление заказа
15	Афиша студенческих мероприятий	Регистрация на событие, электронный билет с QR-кодом

4. Задания по видам работ

Задания выполняются последовательно — каждое опирается на предыдущее. Часы соответствуют тематическому плану рабочей программы УП.08.

Задание 1. Техническое задание. Логическая и физическая структура сайта

Объем: 6 ч. **Цель:** спроектировать приложение до того, как начать его рисовать и программировать.

Что нужно сделать

1. Выбрать тему и описать проект: назначение, целевая аудитория (2 портрета пользователя — «персоны»), основные сценарии использования.
2. Проанализировать 3 сайта-конкурента: что у них хорошо, что плохо, что вы сделаете лучше (таблица сравнения).
3. Составить техническое задание: цели, функциональные требования (список возможностей), нефункциональные требования (адаптивность, скорость, поддерживаемые браузеры), этапы и сроки.



4. Построить **логическую структуру** — карту сайта (sitemap): все страницы и переходы между ними (draw.io, Miro, FigJam).
5. Построить **физическую структуру** — схему папок проекта (frontend / backend / database) и **ER-диаграмму базы данных**: таблицы, поля, типы, связи (dbdiagram.io или draw.io).
6. Выбрать и обосновать стек технологий.
7. Создать репозиторий на GitHub, добавить README.md с описанием проекта, .gitignore и первый коммит.

Технологии и инструменты

Google Docs / Word, draw.io, dbdiagram.io, Miro/FigJam, Git, GitHub.

Что сдать (результат)

- ТЗ (документ), карта сайта, ER-диаграмма.
- Ссылка на репозиторий GitHub с README.

★ Задание со звёздочкой (на «отлично»)

Оформить требования в виде пользовательских историй («Как посетитель, я хочу..., чтобы...») и завести доску задач в GitHub Projects (Kanban).

Задание 2. Разработка макета проекта

Объём: 8 ч. Цель: создать дизайн приложения, по которому дальше будет вестись вёрстка.

Что нужно сделать

1. Собрать мудборд: референсы, настроение, цветовые и шрифтовые решения.
2. Сделать **UI-kit**: палитра (основной, акцентный, фоновые, цвета ошибок/успеха), типографика (заголовки H1–H4, текст), кнопки во всех состояниях (обычная, hover, disabled), поля ввода, карточки, иконки.
3. Нарисовать вайрфреймы (черновые схемы) всех страниц из карты сайта.
4. Сделать детальные макеты: главная страница + не менее 4 внутренних страниц; для каждой — версия для компьютера (1440 px) и для телефона (360–390 px).



5. Использовать Auto Layout, компоненты и стили, чтобы макет было удобно верстать.
6. Собрать кликабельный прототип с переходами между страницами.

Технологии и инструменты

Figma / Pixso / Penpot, Google Fonts, Unsplash, иконки Lucide / Iconify.

Что сдать (результат)

- Ссылка на макет с открытым доступом на просмотр.
- Скриншоты основных экранов в отчёт, ссылка на прототип в README.

★ Задание со звёздочкой (на «отлично»)

Тёмная тема в макете и дизайн-токены (переменные цветов и отступов), которые затем перейдут в CSS-переменные.

Задание 3. Форматирование страниц с помощью CSS. Стартовая страница

Объём: 8 ч. Цель: запустить фронтенд-проект и сверстать стартовую страницу точно по макету.

Что нужно сделать

1. Создать проект фронтенда через Vite (React / Vue / Svelte) и разобраться со структурой папок.
2. Перенести UI-kit в код: CSS-переменные (или конфиг Tailwind) для цветов, шрифтов, отступов, скруглений.
3. Разбить страницу на компоненты: Header, Hero, Card, Button, Footer и т. д.
4. Сверстать стартовую страницу по макету, используя Flexbox и CSS Grid.
5. Сделать адаптивную вёрстку (mobile-first), минимум 3 контрольные точки: телефон, планшет, компьютер.
6. Подключить шрифты и иконки, оптимизировать изображения (формат WebP).



7. **Первый деплой:** опубликовать фронтенд на GitHub Pages или Vercel (автоматически при каждом push в ветку main).

Технологии и инструменты

Node.js, Vite, React / Vue / Svelte, SCSS или Tailwind CSS, DevTools браузер, GitHub Actions / Vercel.

Что сдать (результат)

- Ссылка на опубликованную стартовую страницу.
- Скриншоты: макет и верстка рядом (для сравнения), вид на телефоне.

★ Задание со звёздочкой (на «отлично»)

Переключатель светлой/тёмной темы с сохранением выбора; оценка Lighthouse по разделам Performance и Accessibility не ниже 90.

Задание 4. Гиперссылки и навигация. Меню сайта. Якорные ссылки

Объём: 6 ч. Цель: связать все страницы приложения в удобную навигацию.

Что нужно сделать

1. Подключить маршрутизацию (React Router / Vue Router / SvelteKit) и создать все страницы из карты сайта.
2. Сделать шапку с меню: выделение активного пункта, выпадающее подменю, «бургер»-меню на телефоне.
3. Сделать динамические маршруты для отдельных объектов, например /movies/:id, /products/:id.
4. Добавить хлебные крошки (breadcrumbs) и красиво оформленную страницу 404.
5. На главной странице сделать **якорные ссылки** на разделы (#about, #services, #contacts) с плавной прокруткой и кнопку «Наверх».
6. Оформить подвал (footer) с навигацией, контактами и ссылками на соцсети (внешние ссылки открываются в новой вкладке).

Технологии и инструменты

React Router / Vue Router, CSS scroll-behavior, scroll-margin-top.



Что сдать (результат)

- Работаящая навигация на опубликованном сайте.
- Карта переходов в отчёте.

★ Задание со звёздочкой (на «отлично»)

Подсветка текущего раздела в меню при прокрутке страницы (scrollspy) через Intersection Observer.

Задание 5. Таблицы для организации страниц. Табличный дизайн. Подключение базы данных

Объём: 6 ч. Цель: поднять сервер и базу данных и вывести реальные данные в таблицах и сетках.

Что нужно сделать

1. Создать базу данных по ER-диаграмме из задания 1 (не менее 3 связанных таблиц). Написать миграции и наполнить БД тестовыми данными — не менее 30 записей (seed-скрипт).
2. Создать бэкенд и REST API: получение списка (GET /api/items) и одного объекта (GET /api/items/:id).
3. Получить данные на фронтенде (fetch / axios) и вывести их **в таблице**: сортировка по колонкам, поиск, фильтры, пагинация.
4. Сделать таблицу адаптивной: на телефоне строки превращаются в карточки или появляется горизонтальная прокрутка.
5. Сделать раскладку каталога / галереи на CSS Grid — современная замена «табличной вёрстке» (сравнить подходы в отчёте).
6. Показывать состояния: «Загрузка...», «Ничего не найдено», «Ошибка сервера».

Технологии и инструменты

Бэкенд-фреймворк на выбор, PostgreSQL / MySQL / SQLite, ORM (Prisma, SQLAlchemy, Eloquent), Postman или Thunder Client для проверки API.

Что сдать (результат)

- Репозиторий с кодом бэкенда, миграциями и seed-скриптом.



- Скриншоты таблиц и проверки API в Postman.

★ **Задание со звёздочкой (на «отлично»)**

Панель администратора с таблицей всех записей и экспортом данных в CSV или Excel.

Задание 6. Сбор информации с помощью форм

Объём: 6 ч. Цель: научить приложение принимать данные от пользователя и сохранять их в базу.

Что нужно сделать

1. Сделать регистрацию и вход: пароль хранится в БД только в виде хеша (bcrypt / argon2), авторизация через сессии или JWT.
2. Сделать основную форму проекта (бронирование, заказ, заявка, отзыв) — данные сохраняются в БД (POST), их можно изменить (PUT/PATCH) и удалить (DELETE).
3. Проверять данные **на клиенте** (обязательные поля, формат e-mail и телефона, маски ввода) и **на сервере** (повторная проверка, понятные сообщения об ошибках).
4. Защититься от SQL-инъекций (ORM или параметризованные запросы) и XSS.
5. Сделать загрузку изображения через форму (аватар, фото объявления и т. п.).
6. Опубликовать бэкенд и БД на хостинге. Пароли и ключи хранить в переменных окружения (файл .env не попадает в репозиторий, в репозитории лежит .env.example).

Технологии и инструменты

React Hook Form / VeeValidate / Zod, bcrypt, JWT, переменные окружения, хостинг для бэкенда и БД.

Что сдать (результат)

- Полностью рабочие формы на опубликованном сайте.
- Тестовый аккаунт для проверки (логин и пароль в отчёте).



★ Задание со звёздочкой (на «отлично»)

При новой заявке отправлять уведомление в Telegram-бот администратора или письмо на e-mail.

Задание 7. Анимация на страницах сайта. Слайдер

Объём: 6 ч. Цель: оживить интерфейс и довести проект до финальной версии.

Что нужно сделать

1. Сделать слайдер (Swiper или собственный): автопрокрутка, стрелки, точки, свайп на телефоне. Слайды загружаются из БД.
2. Добавить микроанимации: hover-эффекты кнопок и карточек, анимация открытия меню и модальных окон.
3. Сделать появление блоков при прокрутке (Intersection Observer, AOS, GSAP или Framer Motion).
4. Добавить скелетоны (заглушки) на время загрузки данных и плавные переходы между страницами.
5. Учесть настройку prefers-reduced-motion: у пользователей, отключивших анимации, сайт работает без них.
6. Финальный деплой: проверить сайт на телефоне, в двух браузерах, исправить ошибки, обновить README (описание, стек, скриншоты, ссылка, инструкция запуска).

Технологии и инструменты

CSS transitions / @keyframes, Swiper, GSAP, Framer Motion, AOS, Lottie.

Что сдать (результат)

- Финальная версия проекта по ссылке.
- Короткая видеозапись экрана (30–60 с) с демонстрацией анимаций.

★ Задание со звёздочкой (на «отлично»)

Lottie-анимация, параллакс-эффект или анимация на Canvas / Three.js на главной странице.



5. Оформление отчёта (6 ч)

Отчёт оформляется в формате Word по требованиям колледжа и содержит:

1. Титульный лист, содержание, введение (цель и задачи практики).
2. Техническое задание, карта сайта, ER-диаграмма.
3. Описание и обоснование выбранного стека технологий.
4. Макеты (скриншоты) и сравнение «макет — вёрстка».
5. Описание каждого этапа (задания 3–7): что сделано, скриншоты, фрагменты кода.
6. Описание структуры БД и API (таблица «метод — адрес — назначение»).
7. Описание процесса публикации (деплой) и ссылки: на сайт, репозиторий, макет.
8. Заключение: что получилось, какие трудности были и как их решали, что можно улучшить.

6. Дифференцированный зачёт (6 ч)

Защита проходит в форме собеседования. Студент:

1. Открывает проект по публичной ссылке и демонстрирует основные сценарии (регистрация, работа с данными, формы, адаптив на телефоне).
2. Показывает репозиторий и историю коммитов.
3. Рассказывает про архитектуру: как фронтенд получает данные с сервера, как устроена БД.
4. Отвечает на вопросы по коду и вносит небольшую правку «вживую» (например, изменить цвет кнопки или добавить поле в форму).

7. Требования к репозиторию

- Коммиты — после каждого занятия, сообщения понятные: «Добавлена форма бронирования», а не «fix» или «123».
- README.md: название и описание, стек, скриншоты, ссылка на



сайт, инструкция по запуску.

- В репозитории нет паролей, токенов и папки `node_modules` (`.gitignore` настроен).
- Структура понятна: `frontend/`, `backend/` (или монорепозиторий фреймворка).
- При командной работе — у каждого участника свои коммиты; работа через ветки и Pull Request приветствуется.

8. Критерии оценивания

Оценка	Критерии
«Отлично»	Выполнены все 7 заданий. Есть ТЗ, макет (desktop + mobile), фронтенд на фреймворке, бэкенд, БД (3+ таблицы), регистрация и формы с проверкой, слайдер и анимации. Проект опубликован и стабильно работает по ссылке. Регулярные коммиты, качественный README. Выполнено не менее 2 заданий со звёздочкой. Уверенная защита.
«Хорошо»	Выполнены все задания, есть фронтенд, бэкенд, БД и деплой. Допускаются недочёты: неполная адаптивность, мелкие расхождения с макетом, нет проверки данных на сервере. На защите есть неточности в ответах.
«Удовлетворительно»	Есть макет и работающее приложение с подключённой БД, но часть функций не реализована (например, нет авторизации или анимаций), проект опубликован частично (только фронтенд). Коммиты нерегулярные.
«Неудовлетворительно»	Нет базы данных или бэкенда, сайт сделан на чистом HTML или в конструкторе, проект не опубликован, работа не может быть показана или выполнена не самостоятельно.

9. Полезные ресурсы

- MDN Web Docs — developer.mozilla.org/ru (HTML, CSS, JavaScript).
- Современный учебник JavaScript — learn.javascript.ru.
- Официальная документация: react.dev, vuejs.org, svelte.dev, fastapi.tiangolo.com, laravel.com, expressjs.com, prisma.io.
- Git: git-scm.com/book/ru, интерактивный тренажёр learngitbranching.js.org.



- Figma: help.figma.com, сообщество Figma Community (UI-киты для изучения).
- Немцова Т. И., Назарова Ю. В. Практикум по информатике. Компьютерная графика и web-дизайн. — М.: ФОРУМ: ИНФРА-М, 2021.
- Разработка дизайна веб-приложений: учебник / Т. Мусаева, Е. Поколодина, М. Трифанов и др. — М.: Академия, 2020.

